Python 函数进阶与作用域终极避坑指南

核心避坑心法

1. **带不带括号是两码事**：`func` 是函数对象本身（菜谱），`func()` 是执行函数（炒菜）。
2. **执行顺序不是从上到下**：遇到 `def` 永远跳过内部代码，遇到 `函数名()` 立刻跳进去执行，执行完原路返回。
3. **改变量先申请**：想在函数里修改外面的变量，全局变量用 `global`，嵌套外层变量用 `nonlocal`，且必须在赋值前第一行声明！

模块一：函数可以像变量一样传递（高阶函数）

在 Python 中，函数是一等公民，它和数字、字符串一样，可以被赋值给变量，也可以作为参数传给别人，甚至可以作为结果返回。

1. 传“执行结果” vs 传“函数本身”

代码 A：传递执行结果

```
def func(a):
    print(a)
d = 10
func(print(d))
```

- **解析**：当代码执行到 `func(print(d))` 时，Python 会**先**执行括号里的 `print(10)`，屏幕上打出 `10`。因为 `print()` 函数本身没有返回值（默认返回 `None`），所以接着执行 `func(None)`，屏幕上再打出 `None`。

代码 B：传递函数本体

```
def func(a):
    a()  # 拿到传入的函数，并加上括号唤醒它

def target():
    print("这是目标函数")

func(target) # 注意：这里传的是 target，没有括号！
```

- **解析**：这里把 `target` 这个“菜谱”直接交给了 `func`。`func` 内部拿到菜谱后，通过 `a()` 亲自执行了它。

2. 把函数作为“伴手礼”带回来 (闭包基础)

```
def func():
    def inner():
```

```
        print("这是内层函数")
    return inner  # 把内层函数打包带走（不加括号）

function = func() # function 现在就是 inner 函数了
function()        # 加上括号，立刻唤醒！
```

🪆 模块二：嵌套函数的“盗梦空间”

内层函数必须在外层被显式调用，否则它永远沉睡。

```
def func(a):
    b = pow(a, 2)

    def inner(b):
        c = pow(b, 2)
        print(c)

    inner(b) # 在外层函数的最后，唤醒内层函数

func(100)
```

- **解析**：遇到 `def inner(b):` 时，Python 只是记下逻辑，直接跳过。直到遇到 `inner(b)`，才会真正“下潜”到内层去执行计算。

🌍 模块三：作用域与权限越界 (`global` vs `nonlocal`)

Python 默认遵循“单向玻璃法则”：内层可以看外层的变量，但绝不允许直接修改。想修改，必须申请特权。

1. 经典报错案发现场

```
def func():
    a = 10
    nonlocal a  # ❌ 这里会直接报 SyntaxError
    a = 20
```

- **报错原因**：
  1. `nonlocal` 只能在**嵌套函数**里使用，用来寻找“外层函数”的变量。这里只有一层 `func`，找不到“中间层”。
  2. 必须**先声明，后使用**。不能先写 `a = 10` 创建了本地变量，又突然改口说它是外面的。

2. 特权钥匙对比表

关键字	作用范围	核心特权	使用场景

关键字	作用范围	核心特权	使用场景
`global`	全局（最外层）	允许函数内部修改全局变量	任何函数想修改最外层定义的变量时
`nonlocal`	闭包（外层函数）	允许内层函数修改外层函数的变量	**仅限嵌套函数**，内层想改外层变量时